

CORE CONCEPTS

How Claude Code uses prompt caching

Claude Code manages prompt caching automatically. See why a model switch triggers a slow uncached turn, what `/compact` costs, why CLAUDE.md edits don't apply mid-session, and how to check your cache hit rate.

Prompt caching makes Claude Code faster and more cost-efficient. Without caching, the API would reprocess your full history on every turn. With caching, it reuses what it already processed and only does new work for what changed.

Claude Code handles prompt caching for you, unless you **disable it**. It is still useful to know how prompt caching works, because some actions invalidate the cache and make the next response slower and more expensive while it rebuilds. This page covers which actions those are, why some settings wait for a restart to apply, and how to check cache performance when usage looks high.

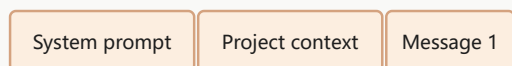
How the cache is organized

Each time you send a message in Claude Code, it makes a new API request. The model doesn't remember anything between requests, so Claude Code re-sends the full context: the system prompt, your project context, every prior message and tool result, and your new message. New content is appended at the end, which means most of each request is identical to the one before it. Prompt caching is how the API avoids reprocessing the part that didn't change.

The API caches by matching the start of each request, called the prefix, against content it recently processed. On a normal turn, the prefix is the entire previous request and only the latest exchange is new. The match is exact, so a change anywhere in the prefix recomputes everything after it. There is no per-file or per-segment caching. See **how prompt caching works** in the API reference for the underlying mechanism.

Each turn resends the full context, and the unchanged prefix reads from cache

Turn 1



First turn: no cache yet, the whole request is processed and written

Turn 2



Read from cache (entire turn 1 request)

New: written to cache

Turn 3



Read from cache (entire turn 2 request)

New

Turn 4: the system prompt changed



Changed

Reprocessed: the prefix no longer matches, so even unchanged content is computed and written again

☐ Cached: read from a previous request ☐ New or recomputed: processed and written to cache

To get the most out of prefix matching, Claude Code orders each request so content that rarely changes between turns comes first:

Layer	Content	Changes when
System prompt	Core instructions, tool definitions, output style	The set of loaded tool definitions changes, or Claude Code is upgraded
Project context	CLAUDE.md, auto memory, unscoped rules	Session starts, or after <code>/clear</code> or <code>/compact</code>
Conversation	Your messages, Claude's responses, tool results	Every turn

A change to the conversation layer leaves the system prompt and project context cached. A change to the system prompt invalidates everything, because all later content now sits behind a different prefix. The third column gives common triggers rather than an exhaustive list, and the sections below cover the full set, including content such as output style that is fixed at session start.

The prefix-match rule explains most of the behaviors on this page. **Plan mode** and **skill loading**, for example, append their instructions as conversation messages, so the cached prefix stays intact.

Two settings aren't part of the prompt text at all, so they don't appear in the layer table, but both are part of the cache key:

- **Model:** each model has its own cache. Switching models recomputes the entire request even when the content is identical. See [Switching models](#) below.
- **Effort level:** each effort level has its own cache for the same model. Changing it mid-session recomputes the entire request, and Claude Code asks you to confirm before applying the change. See [Changing effort level](#) below.

💡 Pick your model and effort level at the top of a session, then save `/compact` for natural breaks between tasks. The fewer changes you make mid-task, the higher your cache hit rate.

Where the cache lives

Caching happens server-side, in whichever infrastructure serves your model. Where that is depends on how you authenticate:

- **API key, Claude subscription, or [Claude Platform on AWS](#):** the cache lives in Anthropic's infrastructure, accessed through the [Claude API](#)
- **Bedrock or Vertex AI:** the cache lives in your cloud provider's serving infrastructure
- **Foundry:** requests route to Anthropic's infrastructure
- **Custom `ANTHROPIC_BASE_URL` or [LLM gateway](#):** the cache lives wherever your requests are forwarded, and whether caching works depends on the gateway

For what each provider stores and processes, see [data usage](#). Wherever the cache lives, entries expire after a period of inactivity, and [Cache lifetime](#) below covers the TTL and how to extend it.

Actions that invalidate the cache

These actions cause the next request to miss part or all of the cache. You see a one-time slower, more expensive turn, after which the new prefix is cached. Most of them are avoidable mid-task once you know they have a cost. A model switch can feel free until you notice the slower turn that follows.

- [Switching models](#)
- [Changing effort level](#)
- [Turning on fast mode](#)
- [Connecting or disconnecting an MCP server](#)
- [Enabling or disabling a plugin](#)
- [Denying an entire tool](#)

- Compacting the conversation
- Upgrading Claude Code

Switching models

Each model has its own cache. Switching with `/model` means the next request reads the entire conversation history with no cache hits, even though the content is identical.

The `opusplan` **model setting** resolves to Opus during plan mode and Sonnet during execution, so each plan-mode toggle is a model switch and starts a fresh cache.

Automatic model fallback on Fable 5 is also a model switch. When a safety classifier flags a request, Claude Code re-runs it on the default Opus model and the session continues there.

Changing effort level

The cache is keyed by **effort level** as well as model, so switching with `/effort` means the next request reads the entire conversation history with no cache hits. Once a conversation has started, Claude Code shows a confirmation dialog before applying an effort change that would invalidate the cache. A change that resolves to the same level already in effect, such as setting the model's default explicitly, skips the dialog and keeps the cache.

Turning on fast mode

Enabling **fast mode** adds a request header that is part of the cache key, so the next request reads the entire conversation history with no cache hits. Those uncached input tokens are billed at **fast mode rates**, which is why turning it on at the start of a session costs less than turning it on deep into a long one. Enabling fast mode from a non-Opus model also **switches your model**, which starts a fresh cache on its own.

The cost applies once per conversation. After the first fast mode turn, Claude Code keeps sending the header and varies only the request's speed setting, which is not part of the cache key. Turning fast mode off, the **automatic fallback to standard speed** after a rate limit, and turning it back on later all keep the cache. `/clear` and `/compact` reset this, since they rebuild the cache at those points anyway.

⚠ Keeping the header across toggles requires Claude Code v2.1.86 or later. On earlier versions, every fast mode toggle and rate-limit fallback invalidates the cache.

Connecting or disconnecting an MCP server

Tool definitions sit in the system prompt layer, so the cache invalidates when the set of tool definitions in the request changes between turns. Toggling the **advisor tool** is an exception: its definition sits after the cache breakpoint, so enabling or disabling `/advisor` keeps the cached prefix intact. Whether an **MCP server** change does this depends on whether its tools are deferred by **tool search** or loaded into the prefix:

- **Deferred tools**, the default on supported models: a server connecting, disconnecting, or changing its tool list only appends new content and doesn't disturb anything already cached.
- **Tools loaded into the prefix**: any change to them invalidates the cache. This happens when **tool search is unavailable or disabled**, such as on Haiku models, on Vertex AI, or with a custom `ANTHROPIC_BASE_URL` gateway. It also happens for a server or tool marked **alwaysLoad**, and for definitions kept upfront by **threshold-based loading**.

When tools load into the prefix, the most common cause of an invalidation is a server connecting or disconnecting mid-session, which can happen without any action on your part: a stdio server's process exits, an HTTP session expires, or a server **reconnects automatically after a transient failure**. A connected server can also push a **dynamic tool update** that changes its tool list.

Editing your MCP config does not by itself change the cache. The new config takes effect only after a restart, which is when the server connects or disconnects.

Enabling or disabling a plugin

Plugins bundle several component types, and the cost of a change depends on which components the plugin provides. Skills, commands, agents, hooks, LSP servers, monitors, and themes never invalidate the cache: anything they add to the request is appended after the existing conversation, so the next request pays for the new content but still reads everything before it from the cache.

The exception is a plugin that provides **MCP servers**. Enabling or disabling one follows the same rules as **connecting or disconnecting an MCP server**: the cache survives when the server's tools are deferred, and the next request re-reads the entire conversation when they load into the prefix.

Plugin changes apply when you run `/reload-plugins` or start a new session. The cost, whether appended announcements or a full re-read, shows up on the first turn after the reload, not when you run `/plugin install`, `/plugin enable`, or `/plugin disable`. As of v2.1.163, when a reload would trigger the full re-read, `/reload-plugins` shows a warning and does not apply the reload. Pass `--force` to apply anyway.

Disabling a plugin you enabled earlier in the session restores the previous request shape. If that prefix is still within its **cache lifetime**, the next request reads the older cache entry instead of rebuilding.

Denying an entire tool

Adding a bare tool name like `Bash` or `WebFetch` as a **deny rule** removes that tool from Claude's context entirely. Built-in tool definitions load into the system prompt layer, so adding or removing one of these rules mid-session invalidates the cache. The change takes effect on the next turn whether you add it through `/permissions` or by **editing a settings file directly**.

Only a deny rule that matches in the tool-name position has this effect: a bare tool name, the equivalent `Bash(*)` form, or a **tool-name glob** like `"*"`. A glob that matches only MCP tools, such as `"mcp__*"`, removes those tools the same way but leaves the cache intact when the matched tools are **deferred**, the default, since deferred definitions were never in the cached prefix. Scoped deny rules like `Bash(rm *)`, and all allow and ask rules, don't change which tools Claude sees. Claude Code checks them when Claude attempts a call, leaving the prefix intact.

Compacting the conversation

Compaction replaces your message history with a summary. By design, this invalidates the conversation layer, since the next request has a new, shorter history that doesn't share a prefix with the old one. Claude Code reuses the system prompt layer and reloads project context from disk, which cache-hits only if `CLAUDE.md` and memory are unchanged since the session started.

To produce the summary, Claude Code sends a one-off request with the same system prompt, tools, and history as your conversation, plus a summarization instruction appended as a final user message. Because it shares your prefix, that request reads the existing cache rather than reprocessing the full history. Most of compaction's time goes to generating the summary, not to a cache miss. The turn that follows rebuilds the conversation cache only for the much shorter summary, so the post-compaction turn is not the slow part.

💡 Compaction works in your favor when the context you discard is content you no longer need. To choose when its overhead happens, run `/compact` at a natural break in your work, such as between tasks, instead of waiting for auto-compaction to trigger mid-task. If you've gone down a path you want to abandon entirely, `/rewind` to an earlier turn instead. Rewinding truncates back to a prefix that is already cached, rather than building a new one as compaction does.

Upgrading Claude Code

A new Claude Code version typically updates the system prompt or tool definitions, so the first request after an upgrade rebuilds the cache from the top. **Auto-update** downloads new versions in the background but applies them on the next launch, never mid-session, so you see this as an uncached first turn after restarting rather than a surprise during a session. Set

`DISABLE_AUTOUPDATER=1` to control when upgrades apply.

❗ **Resuming a session** after an upgrade reprocesses the entire conversation history with no cache hits, since the history now sits behind a different system prompt. The cost scales with how long the resumed conversation is, so the first turn back into a long session can be the most expensive request you send.

Actions that keep the cache

These actions either append to the end of the conversation or don't touch the request at all. Some of them, such as editing CLAUDE.md or changing output style, are also why a setting change waits for a restart to apply.

- Editing files in your repository
- Editing CLAUDE.md mid-session
- Changing output style
- Changing permission mode
- Invoking skills and commands
- Running `/recap`
- Rewinding the conversation
- Spawning a subagent

Editing files in your repository

File contents enter context only when Claude reads them, and reads append to the conversation. Editing a file Claude previously read does not retroactively change the earlier read in history. Instead, Claude Code appends a `<system-reminder>` noting the file changed, and Claude re-reads it if needed.

Editing CLAUDE.md mid-session

Your project-root and user-level CLAUDE.md files are read once at session start and held in memory. Editing them mid-session does not invalidate the cache, but the edit also doesn't apply. Claude keeps working with the version that was loaded at session start. The new content loads on the next `/clear` , `/compact` , or restart.

Nested CLAUDE.md files in subdirectories and **rules with paths: frontmatter** load later, when Claude first reads a matching file. Editing one before it loads does take effect. After it loads, the content is part of the conversation history, so a mid-session edit doesn't retroactively change it.

Changing output style

Output style is part of the system prompt, which Claude Code reads once at session start. Changing it via `/config` or the `outputStyle` setting mid-session does not invalidate the cache, but the change also doesn't apply. Claude keeps using the style that was loaded at session start. The new style loads on the next `/clear` or restart.

Changing permission mode

Switching between **permission modes**, such as from default to accept edits, does not change the system prompt or tool definitions, so mode changes are cache-safe. The exception is plan mode with the **opusplan** model setting, which switches the model between Opus and Sonnet as you enter or leave plan mode. That makes the mode toggle a **model switch**.

Invoking skills and commands

Skills and **commands** inject their instructions as user messages at the point of invocation. Nothing earlier in the conversation changes.

Running `/recap`

/recap generates a summary for display in your terminal. Unlike `/compact` , it appends the summary as command output rather than replacing your message history, so the cached prefix stays intact.

Rewinding the conversation

`/rewind` truncates your conversation back to an earlier turn. The remaining history is the same content the cache was built from at that point, and the system prompt and project context layers are unchanged, so the next request hits the earlier cache entry. Every turn since then has read through that prefix, which kept the entry warm even if the original turn was longer ago than the TTL.

Restoring file checkpoints alongside the conversation has no separate effect on the cache. File contents enter context only when Claude reads them, the same as editing files in your repository.

Cache lifetime

Cached prefixes expire after a period of inactivity. Each request that hits the cache resets the timer, so the cache stays warm as long as you keep working. After a long enough gap, the next request recomputes the full input and re-establishes the cache, which is why the first turn back after stepping away can be noticeably slower.

The time to live (TTL) controls how long a gap the cache survives. The API offers two: a five-minute TTL, and a one-hour TTL that keeps the cache warm through longer breaks but bills cache writes at a higher rate. Claude Code picks the TTL for you based on how you authenticate, and you can override it with environment variables.

On a Claude subscription

On a Claude subscription, Claude Code requests the one-hour TTL automatically. Usage is included in your plan rather than billed per token, so the longer TTL costs you nothing extra and only affects how long your cache stays warm.

If you've gone over your plan's usage limit and Claude Code is drawing on usage credits, you are billed for that usage, so Claude Code automatically drops the TTL to five minutes.

On an API key or third-party provider

On an API key, Bedrock, Vertex, Foundry, or Claude Platform on AWS, you pay the per-token rates, so the TTL stays at the cheaper five minutes by default. To opt into the one-hour TTL, set

```
ENABLE_PROMPT_CACHING_1H=1
```

On Bedrock, prompt caching support, minimum cacheable prefix length, and one-hour TTL availability all vary by model. If cache token counts stay at zero, check supported models, regions,

and limits in the Bedrock documentation.

Override the TTL

Set `FORCE_PROMPT_CACHING_5M=1` to force the five-minute TTL regardless of authentication. This is useful when you're debugging cache behavior, comparing the two TTLs, or overriding an `ENABLE_PROMPT_CACHING_1H` set in managed settings.

Cache scope

In Claude Code, the cache is effectively scoped to one machine and directory. The system prompt embeds the working directory, platform, shell, OS version, and auto-memory paths, so two sessions in different directories build different prefixes and miss each other's cache. That includes worktrees of the same repository, since each worktree has its own working directory.

Sessions you run in parallel in the same directory build matching prefixes and read each other's cache. Sequential sessions share the prefix only when the git status snapshot at startup matches, since the system prompt also captures branch and recent commits.

The underlying API cache is broader. Caches are isolated between organizations, and on some providers, between workspaces within an organization. Within those boundaries, any two requests with the same model and prefix read the same cache. For Agent SDK callers running fleets of automated processes, see improve prompt caching across users and machines to suppress the per-machine sections of the system prompt and share the cache across machines.

Check cache performance

Cache performance shows up as two token counts the API reports on every response. The most direct way to watch them live is a statusline script that reads the `current_usage` object:

Field	Meaning
<code>cache_creation_input_tokens</code>	Tokens written to the cache on this turn, billed at the cache write rate
<code>cache_read_input_tokens</code>	Tokens served from cache on this turn, billed at roughly 10% of the standard input rate

A high read-to-creation ratio means caching is working well. If creation stays high turn after turn,

something is changing in your prefix. The [actions that invalidate the cache](#) section lists the usual causes.

For visibility across an organization, the OpenTelemetry exporter reports cache read and creation tokens per user and session. See [Monitor usage](#) for the metric and event attribute reference.

Subagents and the cache

A [subagent](#) starts its own conversation with its own system prompt and tool set, separate from the parent's. It builds its own cache, starting with no cache hits on its first call and warming up across its own turns. Subagents use the five-minute TTL even on a subscription, since the automatic one-hour TTL applies to the main conversation.

The parent's cache is unaffected. From the parent's side, the subagent's call and result append to the conversation, leaving the parent's prefix intact.

A [fork](#), by contrast, inherits the parent's system prompt, tools, and conversation history exactly, so its first request reads the parent's cache. The compaction summarization call described in [Compacting the conversation](#) uses the same prefix-sharing approach.

Disable prompt caching

Disabling caching is occasionally useful when debugging caching behavior with a specific model or provider. To turn it off, set one of these environment variables to `1` :

Variable	Effect
<code>DISABLE_PROMPT_CACHING</code>	Disable for all models
<code>DISABLE_PROMPT_CACHING_HAIKU</code>	Disable for Haiku only
<code>DISABLE_PROMPT_CACHING_SONNET</code>	Disable for Sonnet only
<code>DISABLE_PROMPT_CACHING_OPUS</code>	Disable for Opus only
<code>DISABLE_PROMPT_CACHING_FABLE</code>	Disable for Fable only

To set caching policy across an organization, put any of these or the [TTL variables](#) in the `env` block of [managed settings](#). For normal use, leave caching enabled.

Related resources

- **Lessons from building Claude Code: Prompt caching is everything**: the design rationale for plan mode, deferred tool loading, and compaction
- **Explore the context window**: what loads into context and when
- **Reduce token usage**: strategies beyond caching for managing context size
- **Track and reduce costs**: cache token tracking and TTL configuration for Agent SDK callers
- **Prompt caching**: the underlying API mechanism, breakpoints, and pricing